

```

def deposit(self): ❸
    print('*** Deposit ***')
    oAccount = self.getUsersAccount()
    depositAmount = input('Please enter amount to deposit: ')
    theBalance = oAccount.deposit(depositAmount)
    print('Deposited:', depositAmount)
    print('Your new balance is:', theBalance)

def withdraw(self): ❹
    print('*** Withdraw ***')
    oAccount = self.getUsersAccount()
    userAmount = input('Please enter the amount to withdraw: ')
    theBalance = oAccount.withdraw(userAmount)
    print('Withdrew:', userAmount)
    print('Your new balance is:', theBalance)

def getInfo(self): ❺
    print('Hours:', self.hours)
    print('Address:', self.address)
    print('Phone:', self.phone)
    print('We currently have', len(self.accountsDict), 'account(s) open.')

# Special method for Bank administrator only
def show(self):
    print('*** Show ***')
    print('(This would typically require an admin password)')
    for userAccountNumber in self.accountsDict:
        oAccount = self.accountsDict[userAccountNumber]
        print('Account:', userAccountNumber)
        oAccount.show()
    print()

```

Listing 4-10: The modified Bank class

The `Bank` class starts with the `__init__()` method ❶ that saves all relevant information in instance variables.

The new `askForValidAccountNumber()` ❷ method is called from a number of other methods to ask the user for an account number and attempts to verify the given number. First it has a `try/except` block ❸ to ensure that the number is an integer. If it isn't, the `except` block detects the error as a `ValueError` exception but reports the error more clearly by raising a custom `AbortTransaction` exception with a descriptive message. Next, it checks to